NAME: VAISHNAVI E *ROLL NO: 231901059*

EX. NO.8

FABRIC NETWORK DEPLOYMENT AND MANAGEMENT

AIM

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

SOFTWARE REQUIREMENTS

- *Kali Linux / Ubuntu*
- *Docker*
- *Docker Compose*
- *Node.js (Version 18 or later)*
- *npm*
- *Git*
- *Hyperledger Fabric Samples*
- *Hyperledger Fabric Binaries*
- *Hyperledger Fabric Docker Images*
- *Visual Studio Code (Optional)*

HARDWARE REQUIREMENTS

- *Processor: Intel Core i3 or above*
- *RAM: Minimum 8 GB*
- *Storage: Minimum 20 GB Free Disk Space*
- *Internet Connection*

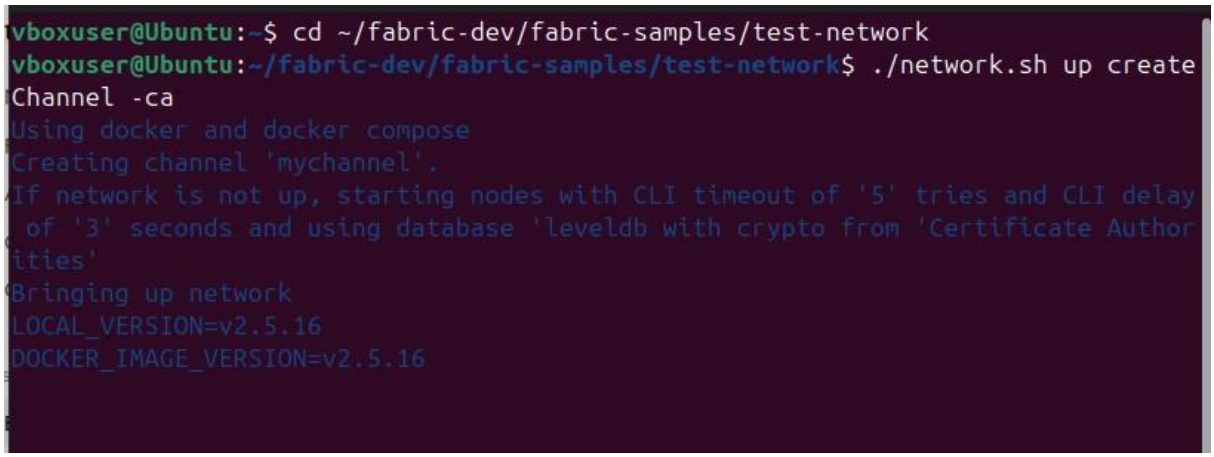
PROCEDURE

Step 1: Navigate to the Fabric Test Network Navigate to the Hyperledger Fabric test network directory. cd ~/fabric-dev/fabric-samples/testnetwork

Step 2: Start the Fabric Network

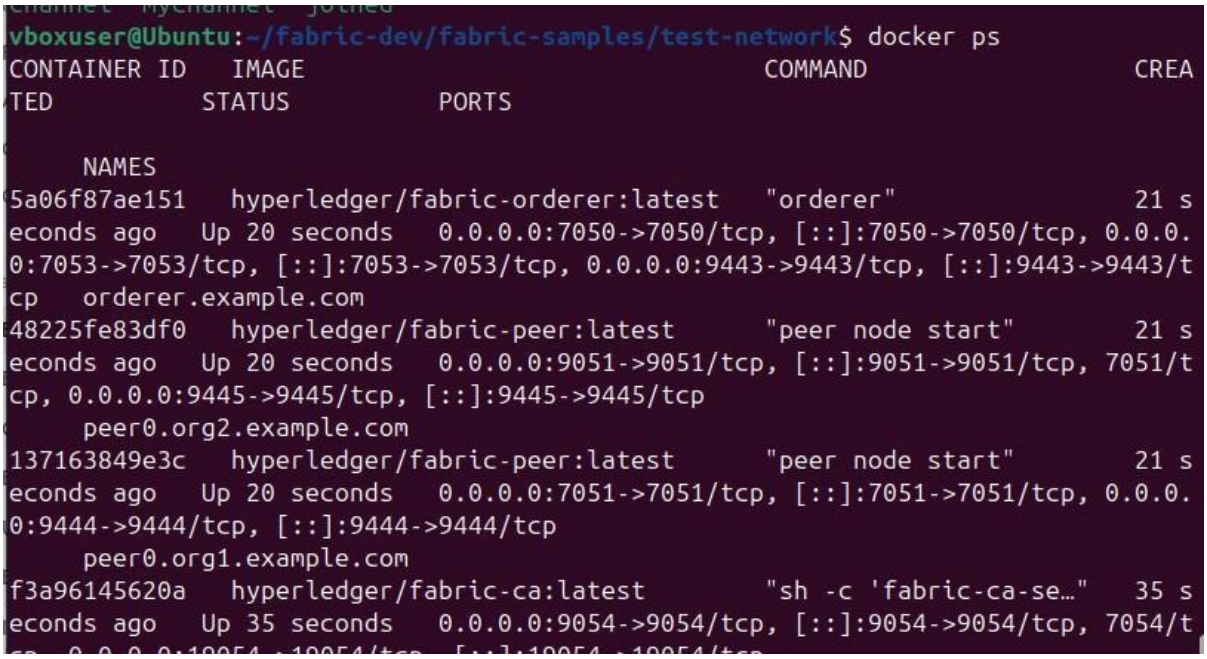
Start the Fabric network, create the application channel, and enable Certificate Authorities.

./network.sh up createChannel -ca



Step 3: Verify Running Docker Containers *Display all currently active Docker containers. docker ps*

This command can be used to verify that the peer, orderer, and Certificate Authority containers are running.

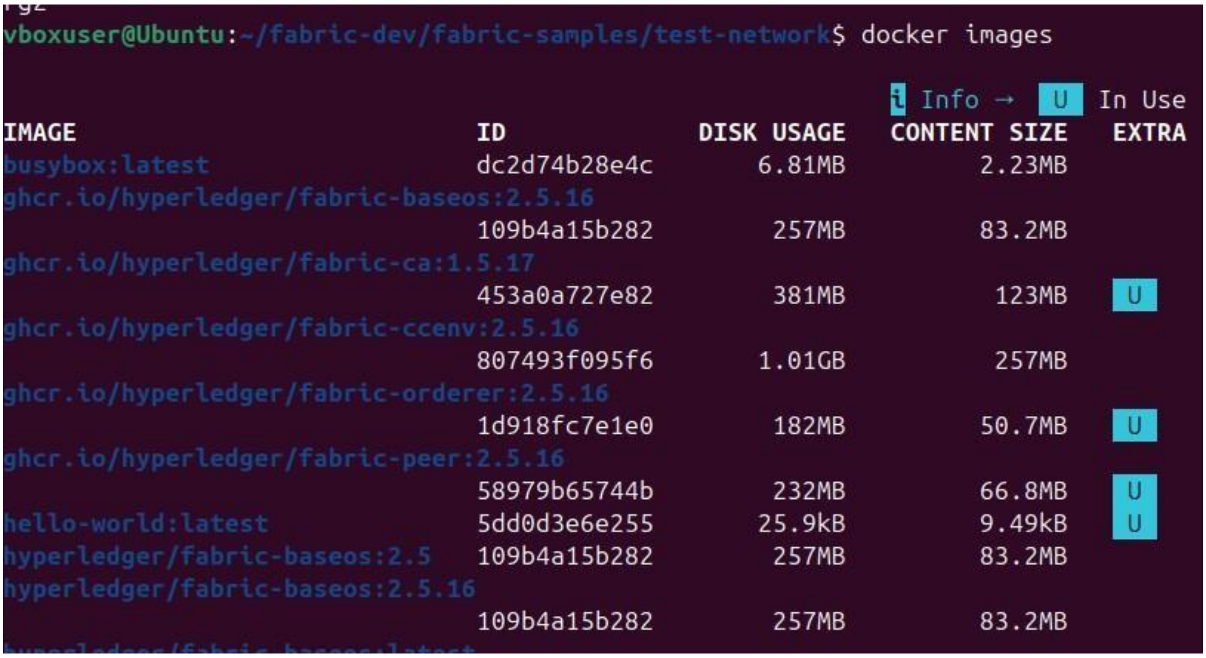


Step 4: Display Docker Images

Display the Docker images available on the system.

docker images

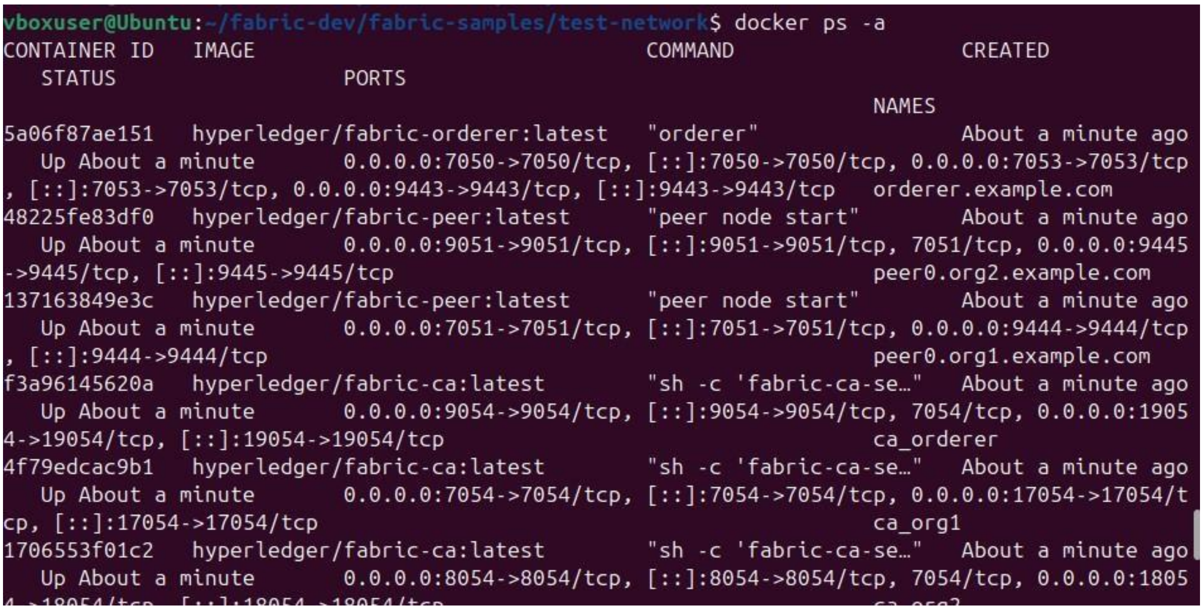
This allows the Fabric Docker images used by the network to be verified.



Step 5: Check Network Status

Display all Docker containers, including stopped containers. docker ps -a

This command helps monitor the current status of Fabric network containers.



Step 6: Deploy Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode to the Fabric network.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../assettransferbasic/chaincode-javascript

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascrip
t -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
```

Step 7: Verify Installed Chaincode *Query the*

installed chaincode packages. *peer*

lifecycle

chaincode queryinstalled

This command verifies that the chaincode package has been installed on the peer.

Step 8: Monitor Peer Logs

Display the logs generated by the Org1 peer node.

docker logs peer0.org1.example.com

The logs can be used to monitor peer startup, connections, transactions, and chaincode-related activities.

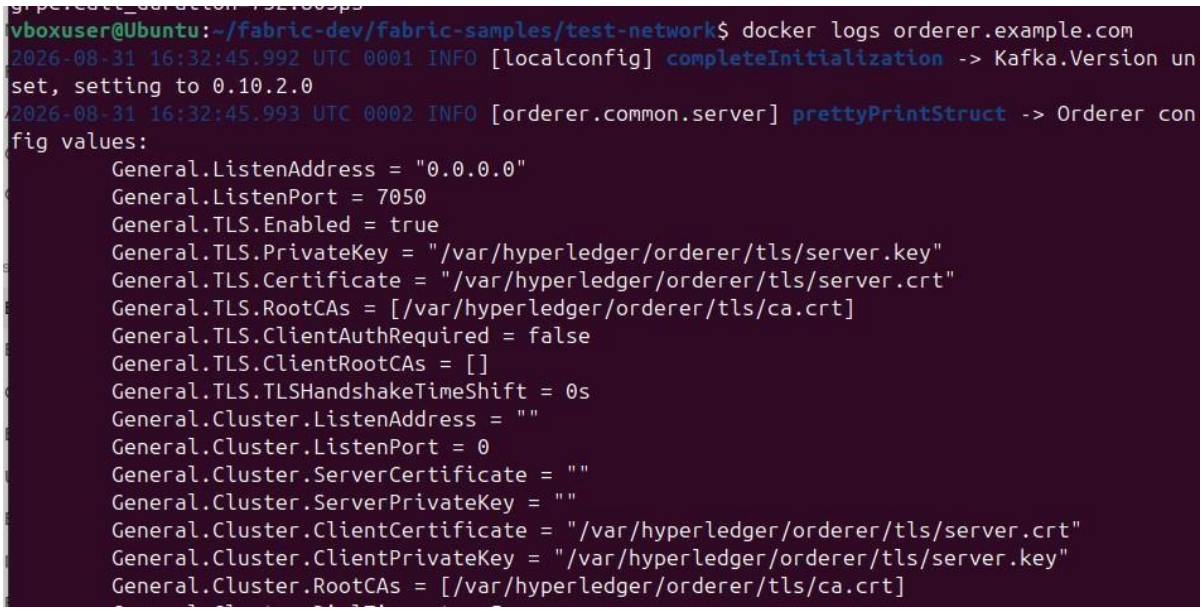
```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode queryinstal
led
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629, Label:
 basic_1.0
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs peer0.org1.example.com
2026-08-31 16:32:45.136 UTC 0001 INFO [nodeCmd] serve -> Starting peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
2026-08-31 16:32:45.141 UTC 0002 INFO [nodeCmd] serve -> Peer config with combined core.yaml s
ettings and environment variable overrides:
chaincode:
  builder: $(DOCKER_NS)/fabric-ccenv:$(TWO_DIGIT_VERSION)
  executetimeout: 300s
  externalbuilders:
```

Step 9: Monitor Orderer Logs

Display the logs generated by the orderer node.

docker logs orderer.example.com

The logs can be used to monitor the ordering service and transaction processing.



Step 10: Verify the Channel *List the*

channels joined by the peer. peer

channel list

This verifies that mychannel has been successfully created and joined by the peer.

Step 11: Stop the Fabric Network

Stop and remove the Fabric network.

./network.sh down

This removes the Fabric containers, network, and generated network artifacts.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-08-31 16:39:48.792 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer conn
ections initialized
Channels peers has joined:
mychannel
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 3/6
 ⠿ Container ca_org1                Stopping                                              2.0s
 ✔ Container peer0.org1.example.com Removed                                               1.7s
 ✔ Container peer0.org2.example.com Removed                                               1.8s
 ⠿ Container ca_org2                Stopping                                              2.0s
 ⠿ Container ca_orderer             Stopping                                              2.0s
 ✔ Container orderer.example.com    Removed                                               1.2s
```

Step 12: Clean Docker Resources

Optionally remove unused Docker containers, networks, images, and other resources. docker system prune -f

This command helps reclaim unused Docker resources and disk space.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker system prune -f
Deleted Containers:
4a6cedf6a948d1759e031f6e82e945acc0df6eb44d6522398904d9c911b0966f
5fff00374402057deb1eb301f9862b2bfe4ddff6abfd97e90fd35f1b9e0ded9e
fab10d165a35b7b84e12a63267468588994df14c99c9969b122b3cd53e2e9715
c848a85aa9cdfe9e0bffc7213eb83bc67e99c68af52eb501f065951f39c2c4e4
74a0cdba564892b8daf6f98b08439cd60fe670e294c2e74b5e16c2e8827af82c
21a81efbebe635cd4d0c7fbbd7aefc65aa521664353d628702b76c90c663cf9d
2c4df2621ee55d9d1e6d0eacb8051d5d67e33fc4c632ee59e083b69a4ea08905
022bb13a6257d24c24d464eae8d52a2fb1f4e9ec501bc0fa0f68ab07e3094240
71e6a51cd89978e9b72aa305d72b41e08fd049a4a4d479d5def250828afa3767
f4cfe3a8fc8c98d438b03253792785e60ec8d26f5f231d923cac86365c0daf8d
371e53b7ed5ced922b84068eb8db73eadfb6d14557acead9f066477be769952e
ae6daf8406e38cb66e273fd7bb1b843ea24035174b908706aab43916ad395a83
5d3201051db4744924f566254e23388709a856855a4d6291589e8ef5e442723e
```

NETWORK MANAGEMENT WORKFLOW

Install Dependencies

|

▼

Start Fabric Network

|

▼

Create Channel

|

▼

Verify Docker Containers

RESULT

The Hyperledger Fabric network was successfully deployed and managed. The peer nodes, ordering service, Certificate Authorities, Docker containers, and application channel were verified. JavaScript chaincode was deployed and its installation was verified. Peer and orderer logs were monitored, and the network was successfully stopped and cleaned up.